

MailForge v1

Engineering Project Plan — Backend Architecture & Core Infrastructure

1. Product Summary

MailForge is a highly optimized, developer-first transactional email API service designed for seamless integration. It handles isolated email infrastructure actions dynamically via programmatically validated keys.

- Create accounts and manage secure profiles.
- Generate and securely hash live API keys.
- Send transactional emails with low latency via standard HTTP requests.
- Maintain accurate tracking logs for delivered, cached, or failed payloads.
- Instantly revoke keys to guarantee security boundaries.

Architectural Scope: MailForge is distinctly built as an infrastructure-oriented developer service, not a marketing or campaign tool. Reliability, validation speed, and atomic execution are primary design goals.

2. Core User Flow

Step 1 — Signup

The user registers their tenant profile via standard credential structures (`Email + Password`). The application backend salts and hashes the credentials before database persistence.

Step 2 — Login

Upon authenticating successfully, the server signs a JSON Web Token (JWT). The client intercepts this token to authenticate subsequent session-based actions across the administrative user interface dashboard.

Step 3 — Create API Key

Inside the authenticated section, the user requests a brand-new workspace key. The application generates a crypto-secure token prefixed format (e.g., `mk_live_83hd9d...`). The unhashed string is exposed exactly **once** to the calling user, while a cryptographic SHA-256 hash is safely stored in the database.

Step 4 — Send Email via API

A client application executes an authenticated request against the API endpoint:

```
POST /api/v1/emails/send
Authorization: Bearer API_KEY
```

The system intercepts the key header, performs immediate atomic lookup and verification, forwards the validated payload downstream to the **Resend** infrastructure layer, records an asynchronous audit status log, and finishes by returning an instantaneous `202 Accepted` or `200 OK` confirmation response frame.

Step 5 — View Email History

The user loads their developer metrics screen to read real-time processing statuses, tracking destinations, and transaction timelines extracted directly from internal system tables.

Step 6 — Revoke API Key

When a key boundary compromise is suspected, the user invalidates it. The record state switches instantly inside the cache/DB layers, terminating any pending or sequential inbound API request workflows signed under that context block.

3. System Architecture

The application architecture maps linear execution blocks to protect underlying services from downstream bottleneck points:

```
graph TD; A[Client Frontend / External API Consumer] --> B[Express.js API Router Infrastructure]; B --> C[Authentication Middleware Context Layer JWT Validation / API Key Cryptography Verification]; C --> D[Isolated Domain Business Logic Service Layer Matrix]; D --> E[Prisma ORM Data Connector]; E --> F[PostgreSQL Relational Database Instance]; F --> G[External Delivery Aggregator Resend Email Core Engine];
```

Client (Frontend / External API Consumer)

↓

Express.js API Router Infrastructure

↓

Authentication Middleware Context Layer (JWT Validation / API Key Cryptography Verification)

↓

Isolated Domain Business Logic (Service Layer Matrix)

↓

Prisma ORM Data Connector

↓

PostgreSQL Relational Database Instance

↓

External Delivery Aggregator (Resend Email Core Engine)

4. Database Design (Prisma Schema Models)

The backend applies a clean relational layout using precise inverse referencing schema fields:

```

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  passwordHash String
  createdAt   DateTime     @default(now())
  apiKeys     ApiKey[]
  emails      EmailLog[]
}

model ApiKey {
  id          String      @id @default(cuid())
  name        String
  keyHash     String
  revoked     Boolean     @default(false)
  createdAt   DateTime     @default(now())
  userId      String
  user        User        @relation(fields: [userId], references: [id])
}

model EmailLog {
  id          String      @id @default(cuid())
  to          String
  subject     String
  status      String      // "sent" | "failed"
  providerId  String?
  createdAt   DateTime     @default(now())
  userId      String
  user        User        @relation(fields: [userId], references: [id])
}

```

5. API Design (Endpoints Layout)

Auth Resource Block

Method	Route Path	Authentication Scope	Payload Structure Example
POST	/api/v1/auth/register	Public	<code>{"email": "test@mail.com", "password": "..."} </code>
POST	/api/v1/auth/login	Public	Returns: <code>{"token": "jwt_token_here"} </code>

API Keys Resource Block

Method	Route Path	Authentication Scope	Payload / Context Description
POST	/api/v1/keys	Bearer JWT	<code>{"name": "Production Key"}</code>
GET	/api/v1/keys	Bearer JWT	Fetches list of existing non-revoked reference labels.
DELETE	/api/v1/keys/:id	Bearer JWT	Marks targeting key identifier state as <code>revoked = true</code> .

Email Core Resource Block

Method	Route Path	Authentication Scope	Payload Execution Structure
POST	/api/v1/emails/send	Bearer API_KEY	<code>{"to": "user@gmail.com", "subject": "Hello", "html": "..."} </code>
GET	/api/v1/emails	Bearer JWT	Returns unified chronological collection list arrays.

6. Middleware Design

1. JWT Auth Middleware

Intercepts traffic hitting dashboard analytics lines (`/keys` , `/emails` history requests). Resolves tokens, extracts claims structure, verifies integrity thresholds against local signature salts, and appends the `userId` parameter into the global execution sequence.

2. API Key Verification Middleware

Intercepts downstream `/v1/emails/send` pipelines. It slices tokens out of incoming transmission payloads, safely applies local system hashes, evaluates equivalency indexes against database tables, and throws strict catch signals if the key has been turned off or deleted.

3. Performance Rate Limiter

Implements protective traffic ceilings set at `100 ext{ requests } / ext{ hour }` mapped against single tracked keys. This cleanly blocks cascading loop overflows, intentional exploitation vectors, and external network resource execution drains.

7. Email Service Layer Isolation

Keeps operational processing isolated cleanly away from HTTP transportation configurations. The function block signature `sendEmail(to, subject, html)` isolates provider interaction abstractions completely. This step validates incoming parameters, establishes connection configurations with the third-party **Resend SDK** engine, handles failures, and drops metadata transactions back into database processing loops before closing out.

8. Error Handling Rules Matrix

Standard response handling components follow rigid structure parameters to simplify consumer handling code setups:

- **401 Unauthorized:** Missing signature keys or damaged structural credential sets.
- **400 Bad Request:** Structural data schema failures, malformed target email definitions, or empty payload parameter arrays.
- **403 Forbidden:** Key hashes matched correctly but reference logs indicate explicit status revocation updates.
- **500 Internal Server Error:** External service dependency dropped connections or cloud layer operational timeout failures.

9. Architectural Folder Layout

```
src/
├─ controllers/      # Extract payload arguments, delegate to business layer, map
responses
├─ routes/          # Core HTTP end route mapping definitions and pipeline
declarations
├─ middleware/      # Strategic runtime intercepts (Authentication and Security
rules)
├─ services/        # True domain core logic processing wrappers
│   └─ email.service.js
├─ utils/           # SHA-256 cryptographic tools and general text formatters
├─ validators/      # Rigid validation structures (Zod schemas validation blocks)
├─ app.js           # Unified pipeline state initialization block
└─ server.js        # Active network interface listener connector ports binding line
```

10. Implementation Phases Roadmap

Phase 1 — Foundation Setup

Initialize Express.js workspace engines, configure connection states with PostgreSQL engines via Prisma ORM maps, and deploy structural hashing loops for administrative registry routes.

Phase 2 — Key Infrastructure

Deploy token generator modules, map secure storage paths inside persistent lookup repositories, and build checking runtime filters for client connections.

Phase 3 — Transaction Routing

Integrate the Resend API provider, declare the inbound payload router path, and hook asynchronous event log generation models up to persistent tracking history components.

Phase 4 — Dashboard Analytics Support

Configure telemetry route calls to fetch logging histories and structure key suspension actions cleanly.

Phase 5 — System Hardening

Bind rate-limiting filters across outer network interfaces, force schema checks on request arguments via strict Zod layers, handle system errors gracefully, and document endpoint signatures cleanly.

11. Engineering Definition of Done

- ✓ Administrative tenants successfully process credential registries and capture persistent access sessions.
- ✓ Dashboard engines accurately initialize production-grade workspace key strings and return safe validation hashes.
- ✓ External system applications cleanly trigger end-to-end email delivery workflows using header key tokens.
- ✓ Transaction records reliably append metadata logs tracking delivered statuses inside database tables.
- ✓ Revocation triggers drop targeted key verification scopes immediately on live traffic tests.
- ✓ Telemetry histories properly display chronological analytics lists down to administrative web interfaces.
- ✓ Runtime parameter faults handle edge-case tracking variables safely without revealing backend structural stack logs.

12. Engineering Competencies Proved

Completing this project provides definitive verification of advanced production-ready core engineering metrics:

- **State-of-the-art Identity Management:** Multi-tiered authorization handling paths utilizing both stateless transient variables (JWTs) and long-lived system tokens (API Keys).
- **Data Pipeline Engineering:** Intentional schema structure abstractions protecting internal indices, database state properties, and transaction historical tracking maps.

- **Third-party System Design:** Asynchronous communication safety boundaries managing runtime constraints, vendor API dependencies, and high-volume data loops.